

LAPORAN TUGAS BESAR IF4044 BIG DATA



Disusun oleh :

Bryan Cornelius Lauwrence	13522033
Bob Kunanda	13523086
Nathaniel Jonathan Rusli	13523013

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132

2026

Daftar Isi

Daftar Isi.....	2
Daftar Gambar.....	3
Daftar Tabel.....	4
BAB I	
DESKRIPSI PERSOALAN.....	5
BAB II	
PEMBAHASAN.....	6
2.1. Arsitektur Sistem dan Alur Kerja Umum.....	6
2.1.1. Alur Kerja Sistem.....	6
2.1.2. Diagram Arsitektur.....	6
2.2. Penjelasan Komponen dan Pemilihan Teknologi.....	7
2.2.1. Data Generator / Source.....	7
2.2.2. Data Lakehouse dan Layer Storage.....	7
2.2.3. Data Processing.....	8
2.2.4. Data Orchestration.....	8
2.2.5. Konfigurasi Komponen.....	8
2.3. Implementasi Langkah Data Processing.....	11
2.3.1. Generate Data CSV.....	11
2.3.2. Proses Load Data ke Lakehouse.....	12
2.3.3. Transformasi Data dengan dbt.....	14
BAB III	
EVALUASI DAN LESSON LEARNED.....	17
3.1. Efisiensi Penyimpanan Data.....	17
3.2. Performa Query Analytics.....	17
3.3. Implementasi Software Engineering pada dbt.....	19
3.4. Tantangan Kompatibilitas Versi Komponen pada Integrasi Sistem.....	20
BAB IV	
LAMPIRAN.....	21
4.1. Repository GitHub.....	21

Daftar Gambar

Gambar 2.1.2.1 Diagram Arsitektur Keseluruhan Proyek.....	7
Gambar 2.2.5.1 Konfigurasi MinIO.....	9
Gambar 2.2.5.2 Pembuatan Bucket MinIO.....	9
Gambar 2.2.5.3 Container Spark.....	10
Gambar 2.2.5.4 Instalasi Iceberg Melalui JARS.....	10
Gambar 2.2.5.5 Koneksi Spark ke Iceberg Pada File Konfigurasi.....	10
Gambar 2.2.5.6 Konfigurasi dbt_project.yml (kiri) dan profiles.yml (kanan).....	11
Gambar 2.3.1.1 Clone DBGEN.....	11
Gambar 2.3.1.2 DBGEN dijalankan dengan parameter scale factor.....	11
Gambar 2.3.1.3 DBGEN membangkitkan 8 file .tbl.....	12
Gambar 2.3.1.3 Upload Dataset ke MinIO.....	12
Gambar 2.3.2.1 Download JAR library untuk Spark.....	12
Gambar 2.3.2.2 Menjalankan Docker.....	13
Gambar 2.3.2.3 Upload CSV ke MinIO.....	13
Gambar 2.3.2.4 Upload Menggunakan MinIO Client Melalui Docker.....	13
Gambar 2.3.2.5 Transformasi CSV ke Parquet Iceberg.....	14
Gambar 2.3.3.1 Test Koneksi Spark Thrift Server.....	14
Gambar 2.3.3.2 Transformasi Data.....	15
Gambar 2.3.3.3 Pengujian Kualitas Data.....	16
Gambar 3.1.1 Perbandingan Ukuran Data.....	17

Daftar Tabel

Tabel 3.2.1 Hasil Benchmark.....	19
----------------------------------	----

BAB I

DESKRIPSI PERSOALAN

Proyek ini berfokus pada penyelesaian masalah inefisiensi penyimpanan dan lambatnya performa *query* analitik pada data CSV mentah dengan cara merancang data *pipeline* yang mencakup integrasi komponen Data Lakehouse, Data Orchestration, dan Data Mart. Untuk mendukung keperluan data proyek, *script* DBGEN digunakan untuk menghasilkan dataset *benchmark* berukuran 10 GB, yang kemudian dimuat ke dalam *data storage* terdistribusi untuk dibentuk menjadi Data Lakehouse (menggunakan Apache Iceberg berformat Parquet) hingga pemrosesan transformasi data menggunakan dbt untuk menghasilkan Data Mart. Melalui implementasi *end-to-end* proyek, dokumen ini akan menganalisis perbandingan ukuran data dan kecepatan eksekusi *simple group-by query* antara CSV langsung terhadap Data Lakehouse, sekaligus mendemonstrasikan penerapan prinsip *software engineering* menggunakan fitur-fitur dbt agar kode transformasi data yang dihasilkan menjadi lebih *readable*, *maintainable*, dan *reusable*.

BAB II

PEMBAHASAN

2.1. Arsitektur Sistem dan Alur Kerja Umum

Proyek ini membangun arsitektur *big data end-to-end* berbasis konsep *lakehouse*. Data benchmark TPC-H dibangkitkan menggunakan DBGEN dalam format .tbl (teks dengan delimiter |), disimpan pada *object storage* MinIO (*S3-compatible*), kemudian diproses dengan Apache Spark untuk dimuat ke Apache Iceberg sebagai Data Lakehouse dalam format Parquet. Selanjutnya, dbt-spark digunakan untuk melakukan transformasi menjadi Data Mart serta menjalankan data *quality tests* (dbt test).

Arsitektur ini mendemonstrasikan praktik umum *data platform*: pemisahan *layer raw* (tpch-raw) dan *curated/lakehouse* (Iceberg), serta penggunaan tool transformasi (dbt) untuk membangun *mart* yang *reusable* dan terdokumentasi.

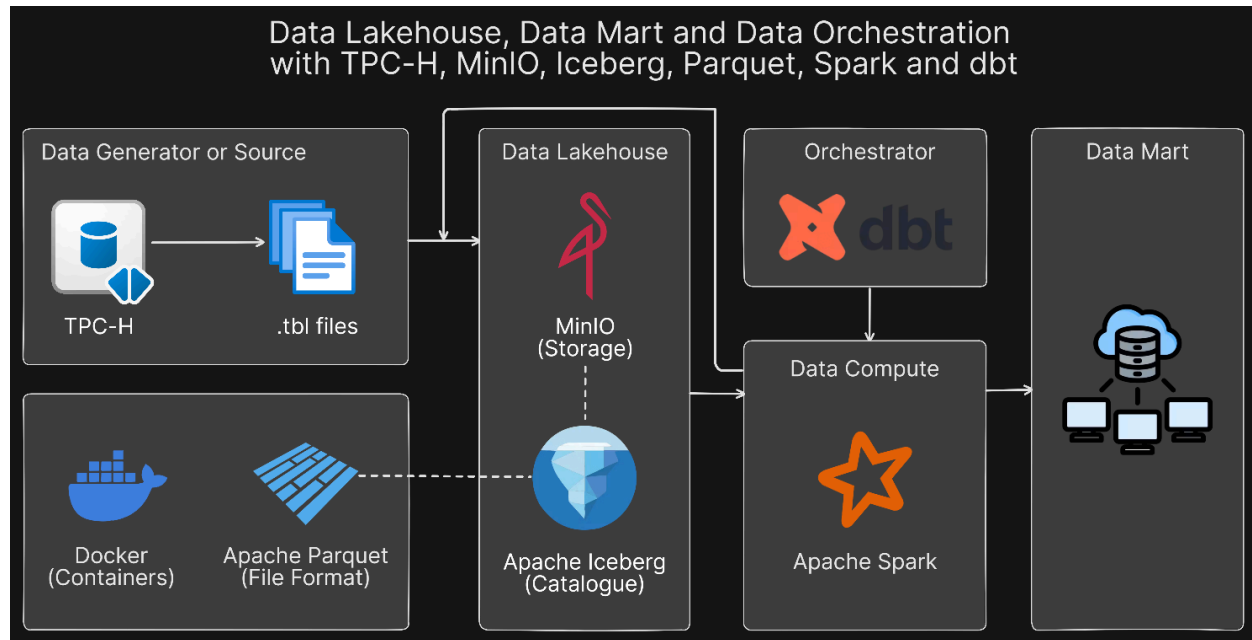
2.1.1. Alur Kerja Sistem

Tahapan alur kerja sistem yang dibuat sebagai berikut:

1. Generate data TPC-H menggunakan DBGEN (output .tbl).
2. Upload file .tbl ke MinIO bucket tpch-raw.
3. Spark membaca .tbl dari MinIO (s3a://tpch-raw/) dan menulis ulang menjadi tabel Iceberg (format Parquet + Snappy) ke *bucket lakehouse*.
4. Validasi: membaca kembali tabel Iceberg, cek row count, dan membandingkan ukuran storage CSV(.tbl) vs Parquet.
5. dbt menjalankan transformasi (minimal 1 query join) dari Lakehouse menjadi Data Mart (Iceberg) dan menjalankan dbt test untuk data quality.

2.1.2. Diagram Arsitektur

Berikut adalah gambar diagram arsitektur dari keseluruhan proses dan *tech stack* yang digunakan pada proyek ini:



Gambar 2.1.2.1 Diagram Arsitektur Keseluruhan Proyek

2.2. Penjelasan Komponen dan Pemilihan Teknologi

2.2.1. *Data Generator / Source*

Data diperoleh dari DBGEN TPC-H Script pada pranala GitHub [berikut](#). DBGEN tersebut dipilih karena mudah digunakan dan tersedia petunjuk penggunaan untuk menghasilkan data yang diperlukan, yaitu data mentah dalam bentuk file CSV ataupun TBL yang secara efektif mensimulasikan skenario *data ingestion* dari sistem operasional tradisional ke dalam ekosistem *big data*. Selain itu, DBGEN yang dipilih mampu mengatur besar data yang dihasilkan (hingga 10 GB) sehingga mempermudah proses pembangunan dan pengujian.

2.2.2. *Data Lakehouse dan Layer Storage*

Data Lakehouse yang digunakan adalah Apache Iceberg karena mendukung dan menjamin karakteristik ACID (Atomicity, Consistency, Isolation, Durability). Pada proyek ini, data akan dibaca berulang kali oleh dbt sehingga Apache Iceberg memastikan konsistensi *read* dan *write* yang dilakukan pada dbt. Selain itu, Apache Iceberg mendukung format penyimpanan Parquet secara *default*, di mana format *columnar* ini sangat penting untuk mengompresi ukuran data ketika dimuat ke Data Lakehouse serta mempercepat *query* analitik secara signifikan.

Layer storage yang digunakan adalah MinIO karena memiliki kompatibilitas dengan S3 API sehingga cocok untuk proses pembelajaran dan juga merupakan standar industri yang sangat relevan untuk diimplementasikan sebelum migrasi ke layanan *cloud* komersial. Selain itu, MinIO menyediakan banyak fitur secara gratis serta dapat di-*host* pada mesin lokal. Adanya *built-in website* pada MinIO pun membantu melihat *bucket* dengan mudah.

2.2.3. Data Processing

Apache Spark dipilih untuk proses *data processing* karena familiaritas dari *tool* ini yang dilengkapi dengan Catalyst Optimizer sehingga akan mempermudah proses pengerjaan proyek. Apache Spark pun secara otomatis membagi pekerjaan ke *spark worker* lainnya tanpa perlu konfigurasi khusus serta mendukung *query* data dengan SQL. Selain itu, *data processing* dibantu dengan penggunaan Jupyter Notebook untuk memudahkan akses ke Apache Spark tanpa mengakses Docker secara langsung.

2.2.4. Data Orchestration

Data orchestration yang digunakan adalah dbt yang secara murni menggunakan SQL sehingga mengurangi beban pembelajaran *framework* baru. dbt juga mampu mengatur dokumentasi yang ditulis sekali saja dan memiliki kelebihan untuk referensi sehingga dbt mendukung modularitas dari data yang beragam. Metadata yang diperlukan juga dapat didefinisikan di awal sehingga mengurangi kemungkinan adanya data yang inkonsisten. Dengan ini, dapat disimpulkan pemilihan dbt memenuhi *software engineering best practices* dengan validasi skema serta metadatanya hingga tersedianya fasilitas otomatisasi pengujian kualitas data dengan perintah dbt test (misal untuk pengecekan unique atau not_null), memastikan Data Mart yang dihasilkan dapat dipercaya keakuratannya sebelum digunakan untuk kegiatan analisis.

2.2.5. Konfigurasi Komponen

Komponen yang digunakan dikonfigurasi antara lain Docker, *file* konfigurasi, dan *file* JARS:

1. MinIO

Dikonfigurasi melalui Docker. Menggunakan dua port, yaitu port 9000 untuk S3 API dan port 9001 untuk akses *website* MinIO UI.

```

minio:
  image: minio/minio:latest
  container_name: minio
  networks:
    - bigdata-net
  ports:
    - "9000:9000" # S3 API endpoint
    - "9001:9001" # MinIO Console (Web UI)
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin123

```

Gambar 2.2.5.1 Konfigurasi MinIO

Selain itu, Docker pun menginisiasi dua *bucket*, yaitu *tpch-raw* untuk menyimpan data mentah dan *lakehouse* untuk merepresentasikan lakehouse.

```

minio-init:
  image: minio/mc:latest
  container_name: minio-init
  networks:
    - bigdata-net
  depends_on:
    minio:
      condition: service_healthy
  entrypoint: >
  /bin/sh -c "
    mc alias set local http://minio:9000 minioadmin minioadmin123;
    mc mb --ignore-existing local/lakehouse;
    mc mb --ignore-existing local/tpch-raw;
    echo 'Buckets created: lakehouse, tpch-raw';
  "

```

Gambar 2.2.5.2 Pembuatan *Bucket* MinIO

2. Apache Spark

Apache Spark dikonfigurasi oleh Docker dan file konfigurasi. Docker membuat tiga *container* Spark, yaitu master untuk koordinator, worker untuk mengeksekusi query atau pekerjaan lainnya, dan thrift sebagai SQL *endpoint* bagi dbt.

```

spark-master:
  image: apache/spark:3.5.8-scala2.12-java17-python3-r-ubuntu
  container_name: spark-master

```

```

spark-worker:
  image: apache/spark:3.5.8-scala2.12-java17-python3-r-ubuntu
  container_name: spark-worker

```

```

spark-thrift:
  image: apache/spark:3.5.8-scala2.12-java17-python3-r-ubuntu
  container_name: spark-thrift

```

Gambar 2.2.5.3 Container Spark

Di sisi lain, file konfigurasi mengatur perilaku Spark, seperti *core* yang digunakan, jumlah partisi, serta koneksi ke komponen lain (MinIO dan Iceberg).

3. Apache Iceberg

Apache Iceberg dikonfigurasi melalui JARS dan, seperti yang dijelaskan sebelumnya, akan terhubung ke Spark melalui file konfigurasi.

```
echo "Apache Iceberg JARs..."
download_jar "$BASE_MVN/org/apache/iceberg/iceberg-spark-runtime-3.5
```

Gambar 2.2.5.4 Instalasi Iceberg Melalui JARS

```
spark.sql.extensions org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions
spark.sql.catalog.spark_catalog org.apache.iceberg.spark.SparkCatalog
spark.sql.catalog.spark_catalog.type hadoop
spark.sql.catalog.spark_catalog.warehouse s3a://lakehouse/

spark.sql.catalog.local org.apache.iceberg.spark.SparkCatalog
spark.sql.catalog.local.type hadoop
spark.sql.catalog.local.warehouse s3a://lakehouse/
```

Gambar 2.2.5.5 Koneksi Spark ke Iceberg Pada File Konfigurasi

4. dbt (data build tool)

Konfigurasi dbt diatur oleh dua file, yaitu `dbt_project.yml` untuk mendefinisikan aturan transformasi dan `profiles.yml` untuk mengatur koneksi dan environment.

```
tpch:
  target: dev
  outputs:
    dev:
      type: spark
      method: thrift
      host: localhost
      port: 10001
      schema: tpch_mart
      connect_retries: 5
      connect_timeout: 60
      threads: 1

name: 'tpch_pipeline'
version: '1.0.0'
config-version: 2
profile: 'tpch'
model-paths: ["models"]
macro-paths: ["macros"]
test-paths: ["tests"]
analysis-paths: ["analyses"]
target-path: "target"
clean-targets:
  - "target"
models:
  tpch_pipeline:
    staging:
      +materialized: view
    marts:
      +materialized: table
      +file_format: iceberg
```

Gambar 2.2.5.6 Konfigurasi dbt_project.yml (kiri) dan profiles.yml (kanan)

2.3. Implementasi Langkah Data Processing

2.3.1. Generate Data CSV

```
bobkunanda@Bobs-MacBook-Air-3 Kode % git clone https://github.com/gregrahn/tpch-kit.git
cd tpch-kit/dbgen
Cloning into 'tpch-kit'...
remote: Enumerating objects: 8883, done.
remote: Total 8883 (delta 0), reused 0 (delta 0), pack-reused 8883 (from 1)
Receiving objects: 100% (8883/8883), 29.24 MiB | 4.13 MiB/s, done.
Resolving deltas: 100% (1130/1130), done.
bobkunanda@Bobs-MacBook-Air-3 dbgen % make
```

Gambar 2.3.1.1 Clone DBGEN

Tahap awal pada proses ini adalah melakukan *clone repository* DBGEN sebagai sumber official *generator* untuk data TPC-H. DBGEN digunakan karena mampu membangkitkan *dataset* yang bersifat deterministic dan dapat diskalakan sesuai kebutuhan eksperimen. Dengan melakukan *clone* dari *source code* DBGEN, seluruh proses pembuatan data dapat direproduksi secara konsisten pada *environment* yang berbeda.

```
bobkunanda@Bobs-MacBook-Air-3 tpch-kit % ./dbgen -s 10
```

Gambar 2.3.1.2 DBGEN dijalankan dengan parameter *scale factor*

Setelah *repository* DBGEN berhasil di-*clone*, langkah berikutnya adalah menjalankan proses kompilasi dan eksekusi DBGEN dengan parameter *scale factor*. Parameter ini menentukan ukuran data yang akan dibangkitkan, sehingga *volume* dataset dapat disesuaikan dengan kebutuhan pengujian. Dalam implementasi ini, *scale factor* digunakan untuk menghasilkan data TPC-H dalam ukuran yang cukup besar agar cocok digunakan sebagai simulasi data *warehouse* dan *pipeline lakehouse*.

```
bobkunanda@Bobs-MacBook-Air-3 tpch-csv % ls -lh *.tbl
-rw-r--r--@ 1 bobkunanda  staff   232M May 19 16:54 customer.tbl
-rw-r--r--@ 1 bobkunanda  staff    7.2G May 19 16:54 lineitem.tbl
-rw-r--r--@ 1 bobkunanda  staff   2.1K May 19 16:54 nation.tbl
-rw-r--r--@ 1 bobkunanda  staff   1.6G May 19 16:54 orders.tbl
-rw-r--r--@ 1 bobkunanda  staff   230M May 19 16:54 part.tbl
-rw-r--r--@ 1 bobkunanda  staff   1.1G May 19 16:54 partsupp.tbl
-rw-r--r--@ 1 bobkunanda  staff   384B May 19 16:54 region.tbl
-rw-r--r--@ 1 bobkunanda  staff    13M May 19 16:54 supplier.tbl
```

Gambar 2.3.1.3 DBGEN membangkitkan 8 file .tbl

Hasil dari proses DBGEN adalah delapan *file* .tbl yang merepresentasikan tabel-tabel utama TPC-H, yaitu *customer*, *lineitem*, *nation*, *orders*, *part*, *partsupp*, *region*, dan *supplier*. *File* .tbl ini sebenarnya merupakan file CSV dengan *delimiter* |, sehingga masih berstatus data mentah yang perlu diproses lebih lanjut. Pada tahap ini, seluruh data sudah selesai dibangkitkan dan siap untuk dipindahkan ke MinIO sebagai sumber data awal.

```
bobkunanda@Bobs-MacBook-Air-3 Kode % bash scripts/upload-tpch-to-minio.sh

=====
Upload TPC-H Data ke MinIO (tpch-raw)
=====
Source : ./data/tpch-csv
Bucket : tpch-raw
Added 'local' successfully.
Bucket created successfully 'local/tpch-raw'.
'/data/tpch-csv/customer.tbl' -> 'local/tpch-raw/customer.tbl'
'/data/tpch-csv/lineitem.tbl' -> 'local/tpch-raw/lineitem.tbl'
'/data/tpch-csv/nation.tbl' -> 'local/tpch-raw/nation.tbl'
'/data/tpch-csv/orders.tbl' -> 'local/tpch-raw/orders.tbl'
'/data/tpch-csv/part.tbl' -> 'local/tpch-raw/part.tbl'
'/data/tpch-csv/partsupp.tbl' -> 'local/tpch-raw/partsupp.tbl'
'/data/tpch-csv/region.tbl' -> 'local/tpch-raw/region.tbl'
'/data/tpch-csv/supplier.tbl' -> 'local/tpch-raw/supplier.tbl'
mc: <ERROR> Failed to copy '/data/tpch-csv/partsupp.tbl'. The specified multipart upload does not exist. The
alid, or the upload may have been aborted or completed.
mc: <ERROR> Failed to copy '/data/tpch-csv/orders.tbl'. The specified multipart upload does not exist. The u
id, or the upload may have been aborted or completed.
mc: <ERROR> Failed to copy '/data/tpch-csv/lineitem.tbl'. The specified multipart upload does not exist. The
alid, or the upload may have been aborted or completed.
time="2026-05-21T16:12:38+07:00" level=error msg="Error waiting for container: write /var/lib/desktop-contai
```

Gambar 2.3.1.3 Upload Dataset ke MinIO

Setelah seluruh *file* .tbl berhasil dibuat, proses pembangkitan data dinyatakan selesai dan dataset siap diunggah ke MinIO. Tahap ini menandai berakhirnya data generation dan awal dari proses data *ingestion* ke dalam *lakehouse*. Dengan data yang sudah tersedia dalam format *file* mentah, *pipeline* dapat dilanjutkan ke tahap *upload* dan transformasi berikutnya.

2.3.2. Proses Load Data ke Lakehouse

```
bobkunanda@Bobs-MacBook-Air-3 Kode % bash scripts/download-jars.sh
```

Gambar 2.3.2.1 Download JAR library untuk Spark

Sebelum melakukan *load* data ke *lakehouse*, *Apache Spark* perlu dilengkapi dengan beberapa JAR *library* tambahan agar dapat terintegrasi dengan *Iceberg* dan *storage* berbasis S3. JAR ini dibutuhkan untuk mendukung koneksi ke MinIO, pembacaan data dari *object storage*, serta penulisan data ke format *Iceberg*. Tanpa dependensi ini, Spark tidak dapat menjalankan proses transformasi data dengan benar.

```
bobkunanda@Bobs-MacBook-Air-3 Kode % docker compose up -d
```

Gambar 2.3.2.2 Menjalankan Docker

Tahap berikutnya adalah menjalankan Docker *stack* yang berisi komponen utama sistem, seperti Spark, MinIO, dan layanan pendukung lainnya. Docker digunakan untuk memastikan seluruh service berjalan dalam *environment* yang konsisten dan terisolasi. Dengan *stack* yang aktif, proses *load* data dari CSV ke *lakehouse* dapat dilakukan secara terstruktur.

```
bobkunanda@Bobs-MacBook-Air-3 Kode % bash scripts/upload-tpch-to-minio.sh
```

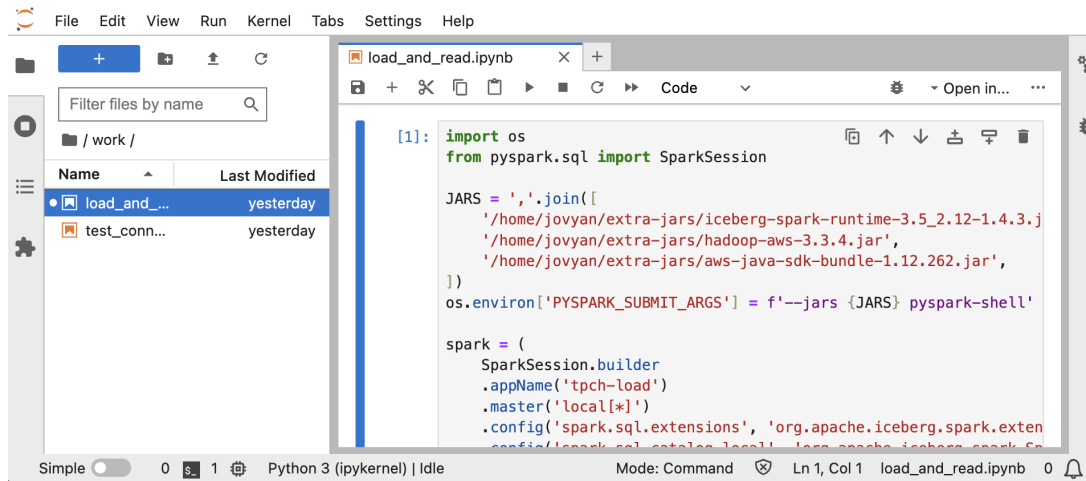
Gambar 2.3.2.3 Upload CSV ke MinIO

Setelah *environment* siap, seluruh *file* .tbl diunggah ke *bucket* tpch-raw pada MinIO menggunakan *script* yang telah disediakan. Bucket ini berfungsi sebagai *staging area* untuk menyimpan data mentah sebelum diproses oleh Spark. Dengan demikian, MinIO menjadi lapisan *storage* awal yang menampung seluruh dataset TPC-H hasil DBGEN.

```
bobkunanda@Bobs-MacBook-Air-3 Kode % docker run --rm \
--network kode_bigdata-net \
-v "$(pwd)/data:/data" \
--entrypoint /bin/sh \
minio/mc:latest \
-c "
mc alias set local http://minio:9000 minioadmin minioadmin123 &&
mc mb --ignore-existing local/tpch-raw &&
mc cp /data/tpch-csv/*.tbl local/tpch-raw/ &&
mc ls local/tpch-raw
"
```

Gambar 2.3.2.4 Upload Menggunakan MinIO Client Melalui Docker

Proses *upload* dilakukan secara otomatis menggunakan MinIO *Client* (mc) yang dijalankan di dalam *container* Docker. Pendekatan ini memudahkan proses *upload* karena tidak memerlukan instalasi MinIO Client secara manual di *host machine*. *Script* akan mengatur alias ke MinIO, memastikan *bucket* tersedia, lalu menyalin seluruh file .tbl ke dalam *object storage*.



Gambar 2.3.2.5 Transformasi CSV ke Parquet Iceberg

Proses upload dilakukan secara otomatis menggunakan MinIO Client (mc) yang dijalankan di dalam *container* Docker. Pendekatan ini memudahkan proses *upload* karena tidak memerlukan instalasi MinIO Client secara manual di *host machine*. *Script* akan mengatur alias ke MinIO, memastikan bucket tersedia, lalu menyalin seluruh *file .tbl* ke dalam *object storage*.

2.3.3. Transformasi Data dengan dbt

```
bobkunanda@Bobs-MacBook-Air-3 dbt % dbt debug
08:47:23 Running with dbt=1.11.8
08:47:23 dbt version: 1.11.8
08:47:23 python version: 3.11.9
08:47:23 python path: /Users/bobkunanda/.pyenv/versions/3.11.9/bin/python3.11
08:47:23 os info: macOS-26.0.1-arm64-arm-64bit
08:47:23 Using profiles dir at /Users/bobkunanda/Documents/Code/Github/Untitled/Proyek-Big-Data-IF4044/Kode/dbt
08:47:23 Using profiles.yml file at /Users/bobkunanda/Documents/Code/Github/Untitled/Proyek-Big-Data-IF4044/Kode/dbt/profiles
08:47:23 Using dbt_project.yml file at /Users/bobkunanda/Documents/Code/Github/Untitled/Proyek-Big-Data-IF4044/Kode/dbt/dbt_p
08:47:23 ect.yml
08:47:23 adapter type: spark
08:47:23 adapter version: 1.8.0
08:47:23 Configuration:
08:47:23   profiles.yml file [OK found and valid]
08:47:23   dbt_project.yml file [OK found and valid]
08:47:23 Required dependencies:
08:47:23   - git [OK found]
08:47:23 Connection:
08:47:23   host: localhost
08:47:23   port: 10001
08:47:23   cluster: None
08:47:23   endpoint: None
08:47:23   schema: tpch_mart
08:47:23   organization: 0
08:47:23 Registered adapter: spark=1.8.0
08:47:23 Connection test: [OK connection ok]
08:47:23 All checks passed!
```

Gambar 2.3.3.1 Test Koneksi Spark Thrift Server

Test koneksi ke Spark Thrift *Server*. Tahap ini dilakukan dengan menjalankan perintah *dbt debug* untuk memastikan dbt dapat terhubung ke Spark Thrift *Server* yang menjadi *layer*

SQL untuk proses transformasi. Pada tahap ini, dbt memeriksa konfigurasi profil, kredensial, *host*, *port*, serta kesesuaian *environment* Python dan *adapter* Spark yang digunakan. Jika hasilnya muncul *All checks passed!*, berarti koneksi sudah valid dan seluruh komponen pendukung seperti Spark Thrift Server, *namespace* Iceberg, dan konfigurasi dbt sudah siap dipakai untuk proses berikutnya.

```
bobkunanda@Bobs-MacBook-Air-3 dbt % dbt run
08:49:27 Running with dbt=1.11.8
08:49:27 Registered adapter: spark=1.8.0
08:49:27 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.tpch_pipeline.staging
08:49:27 Found 1 model, 7 data tests, 5 sources, 504 macros
08:49:27 Concurrency: 1 threads (target='dev')
08:49:27
08:49:28 1 of 1 START sql table model tpch_mart.sales_summary ..... [RUN]
08:52:22 1 of 1 OK created sql table model tpch_mart.sales_summary ..... [OK in 174.13s]
08:52:22
08:52:22 Finished running 1 table model in 0 hours 2 minutes and 54.69 seconds (174.69s).
08:52:22
08:52:22 Completed successfully
08:52:22
08:52:22 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=1
```

Gambar 2.3.3.2 Transformasi Data

Jalankan transformasi. Tahap dbt run digunakan untuk mengeksekusi model transformasi dan membangun *Data Mart* di atas tabel-tabel hasil *load* ke Iceberg. Pada implementasi ini, *model sales_summary* membuat tabel *tpch_mart.sales_summary* di storage MinIO melalui Iceberg, dengan cara menggabungkan lima tabel utama, yaitu *orders*, *lineitem*, *customer*, *nation*, dan *region*. Hasil akhir dari proses ini adalah tabel analitik yang sudah diringkas berdasarkan *region*, *nation*, tanggal order, dan *status*, sehingga data mentah TPC-H berubah menjadi bentuk yang lebih siap untuk analisis bisnis.

```
bobkunanda@Bobs-MacBook-Air-3 dbt % dbt test
08:53:25 Running with dbt=1.11.8
08:53:25 Registered adapter: spark=1.8.0
08:53:25 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.tpch_pipeline.staging
08:53:25 Found 1 model, 7 data tests, 5 sources, 504 macros
08:53:25 Concurrency: 1 threads (target='dev')
08:53:25
08:53:26 1 of 7 START test not_null_sales_summary_nation_name ..... [RUN]
08:53:26 1 of 7 PASS not_null_sales_summary_nation_name ..... [PASS in 0.38s]
08:53:26 2 of 7 START test not_null_sales_summary_o_orderdate ..... [RUN]
08:53:26 2 of 7 PASS not_null_sales_summary_o_orderdate ..... [PASS in 0.50s]
08:53:26 3 of 7 START test not_null_sales_summary_o_orderstatus ..... [RUN]
08:53:27 3 of 7 PASS not_null_sales_summary_o_orderstatus ..... [PASS in 0.20s]
08:53:27 4 of 7 START test not_null_sales_summary_order_count ..... [RUN]
08:53:27 4 of 7 PASS not_null_sales_summary_order_count ..... [PASS in 0.14s]
08:53:27 5 of 7 START test not_null_sales_summary_region_name ..... [RUN]
08:53:27 5 of 7 PASS not_null_sales_summary_region_name ..... [PASS in 0.36s]
08:53:27 6 of 7 START test not_null_sales_summary_total_quantity ..... [RUN]
08:53:27 6 of 7 PASS not_null_sales_summary_total_quantity ..... [PASS in 0.29s]
08:53:27 7 of 7 START test not_null_sales_summary_total_revenue ..... [RUN]
08:53:28 7 of 7 PASS not_null_sales_summary_total_revenue ..... [PASS in 0.14s]
08:53:28
08:53:28 Finished running 7 data tests in 0 hours 0 minutes and 2.34 seconds (2.34s).
08:53:28
08:53:28 Completed successfully
08:53:28
08:53:28 Done. PASS=7 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=7
```

Gambar 2.3.3.3 Pengujian Kualitas Data

Jalankan data quality tests. Setelah model berhasil dibuat, dbt test dijalankan untuk memastikan hasil transformasi memenuhi aturan kualitas data yang telah didefinisikan pada *project*. Pengujian ini biasanya memeriksa atribut seperti not null pada kolom-kolom penting agar tidak ada nilai kosong pada data hasil olahan. Tahap ini penting karena memastikan tabel *Data Mart* yang dihasilkan bukan hanya berhasil dibuat secara teknis, tetapi juga konsisten dan layak digunakan sebagai dasar analisis selanjutnya.

BAB III

EVALUASI DAN *LESSON LEARNED*

3.1. Efisiensi Penyimpanan Data

Berdasarkan percobaan yang telah dilakukan, untuk data yang sama, berkas parquet memiliki ukuran yang lebih kecil dibandingkan berkas awalnya, yaitu CSV (tbl). Perbandingannya dapat dilihat pada Gambar 2.4.1.1.

CSV (tpch-raw)	: 10.38 GB (11,145,550,186 bytes)
Parquet (lakehouse)	: 3.26 GB (3,502,170,493 bytes)
Rasio kompresi	: 3.18x lebih kecil

Gambar 3.1.1 Perbandingan Ukuran Data

Jadi, ukuran berkas parquet berhasil mengurangi 68% ukuran data. Pengurangan tersebut tentunya sangat bermanfaat, khususnya dalam kasus *big data*. Data yang jumlahnya sangat banyak dapat dikurangi ukurannya hingga 70% akan mengurangi keperluan biaya untuk *storage* data, baik biaya pembelian maupun pemeliharaan. Dengan kata lain, berkas parquet menjadi opsi utama untuk menyimpan data dalam suatu arsitektur *big data*.

3.2. Performa *Query Analytics*

Pada benchmark ini digunakan tiga *query* bergaya TPC-H untuk mewakili pola analitik yang umum seperti agregasi sederhana, filter selektif, dan *join multi-tabel*. Ketiga *query* dijalankan pada dua *pipeline* yang sama *engine-nya* (Spark) dan sama *storage-nya* (MinIO), sehingga yang dibandingkan hanya format data. Berikut merupakan *query* yang digunakan untuk *benchmark*.

1. Q1 – *Group-by lineitem* (agregasi sederhana)

```
SELECT
  l_returnflag,
  l_linestatus,
  SUM(l_quantity) AS sum_qty,
  SUM(l_extendedprice) AS sum_base_price,
  AVG(l_discount) AS avg_disc,
  COUNT(*) AS count_order
```

```
FROM {{lineitem}}
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus
```

Query ini melakukan agregasi dan pengelompokan pada tabel *lineitem* untuk menghitung total kuantitas, total harga, rata-rata diskon, dan jumlah baris per kombinasi status pengiriman.

2. Q6 – Filter + *sum* (selektivitas menengah)

```
SELECT
  SUM(l_extendedprice * l_discount) AS revenue
FROM {{lineitem}}
WHERE l_shipdate >= DATE '1994-01-01'
  AND l_shipdate < DATE '1995-01-01'
  AND l_discount BETWEEN 0.05 AND 0.07
  AND l_quantity < 24
```

Query ini menguji performa scan dengan filter tanggal dan nilai diskon, lalu menghitung total *revenue*. Pola ini umum untuk analitik berbasis *time window*.

3. Q3 – *Join* + *top-N* (kompleksitas lebih tinggi)

```
SELECT
  l.l_orderkey AS orderkey,
  SUM(l.l_extendedprice * (1 - l.l_discount)) AS revenue,
  o.o_orderdate AS orderdate,
  o.o_shippriority AS shippriority
FROM {{customer}} c
JOIN {{orders}} o ON c.c_custkey = o.o_custkey
JOIN {{lineitem}} l ON l.l_orderkey = o.o_orderkey
WHERE c.c_mktsegment = 'BUILDING'
  AND o.o_orderdate < DATE '1995-03-15'
  AND l.l_shipdate > DATE '1995-03-15'
GROUP BY l.l_orderkey, o.o_orderdate, o.o_shippriority
ORDER BY revenue DESC, o.o_orderdate
LIMIT 10
```

Query ini melibatkan join tiga tabel untuk menghitung *revenue per order* dan mengambil 10 teratas, mewakili pola analitik yang lebih berat.

Tabel 3.2.1 Hasil Benchmark

Query	CSV avg (s)	Iceberg avg (s)	Speedup
Q1	9.4424	1.3502	6.99x
Q3	22.6146	6.6356	3.41x
Q6	11.5741	1.2382	9.35x

Secara konsisten Iceberg/Parquet lebih cepat dibanding CSV *raw*. Penyebab utamanya adalah format *kolumnar* Parquet yang memungkinkan Spark membaca hanya kolom yang dibutuhkan dan melakukan kompresi yang lebih efektif. CSV membutuhkan parsing teks dan tidak memiliki *metadata layout* yang membantu optimisasi scan. Hasil di atas menunjukkan *speedup* signifikan untuk *query* agregasi dan filter (Q1, Q6), sementara untuk *query join* (Q3) *speedup* tetap terlihat tetapi lebih kecil karena biaya *join* dan *shuffle* tetap dominan.

3.3. Implementasi Software Engineering pada dbt

Penggunaan dbt sebagai *orchestrator end-to-end pipeline* membuat dibutuhkannya adaptasi prinsip *software engineering* yang mencakup:

1. *Separation of concerns*: Struktur direktori dan kode dbt dirancang untuk memisahkan konfigurasi proyek dbt, logika sumber data, proses transformasi, validasi data, hingga pengujian kualitas sehingga modifikasi pada satu bagian tidak akan merusak integritas bagian lainnya.
2. *Reusability*: Pemanfaatan fitur makro berupa Jinja memungkinkan formula kalkulasi yang sering digunakan hanya didefinisikan sekali saja, menghindari *repeated code* dan membuat sentralisasi definisi formula menjadi lebih praktis.
3. *Automated dependency management*: Deklarasi ketergantungan antartabel dan model data dideklarasikan secara eksplisit sehingga alur eksekusi sistem secara otomatis dipastikan selalu benar, menghilangkan potensi *bug* atau *error* bila menggunakan *script* manual.
4. *Code-based documentation*: Deskripsi fungsional masing-masing kolom dan tabel diintegrasikan langsung di dalam *file* konfigurasi, menerapkan *self-documenting code practice* secara otomatis.

5. Pengujian kualitas data yang terintegrasi: Validasi data dengan ketersediaan konfigurasi untuk *command* dbt test telah menjadi bagian langsung dari dbt sehingga proses validasi ini dapat langsung dijalankan untuk memastikan integritas data.

3.4. Tantangan Kompatibilitas Versi Komponen pada Integrasi Sistem

Salah satu aspek terpenting atau yang terpenting adalah *data pipeline system design* dengan menggabungkan berbagai *data engineering* atau *big data tools* sesuai dengan kebutuhan sistem. Proyek ini sendiri telah mengintegrasikan berbagai komponen termasuk MinIO, Apache Spark, Apache Iceberg, hingga dbt. Namun, berdasarkan pengalaman dan tantangan yang dihadapi, terjadi ketidakcocokan versi misalnya pada kasus ini adalah konflik versi antara hadoop-aws dengan Spark Thrift. Dengan ini, setelah mengetahui *tech stack* atau *tools* yang ingin digunakan, penting juga untuk mengetahui dari awal versi yang saling *compatible* satu sama lain.

Agar tidak mengulangi kesalahan dan menghadapi tantangan yang sama kedepannya, langkah preventif utama yang dapat diterapkan adalah penyusunan matriks kompatibilitas secara komprehensif pada fase awal perancangan arsitektur. Matriks ini bertindak sebagai fondasi penting yang memetakan secara presisi irisan versi perangkat lunak yang saling didukung oleh seluruh komponen ekosistem. Di samping itu, keandalan arsitektur ini juga harus didukung oleh upaya pemantauan pembaruan perangkat lunak secara berkelanjutan melalui catatan rilis resmi, serta proses validasi yang ketat di dalam lingkungan pengembangan yang diisolasi. Mengunci definisi versi secara absolut pada file konfigurasi dan melakukan uji coba integrasi berskala kecil akan memberikan kepastian bahwa seluruh infrastruktur data dapat beroperasi secara harmonis tanpa risiko kegagalan akibat pembaruan otomatis yang tidak terprediksi

BAB IV

LAMPIRAN

4.1. Repository GitHub

Pranala menuju Repository GitHub dapat dilihat di [sini](#).